

Additional Experiments:

11a. Programming example for MySQL in Docker

MySQL — SQL Commands in CLI

Step 1: Start MySQL on PWD

```
docker run --name mysql-test -e MYSQL_ROOT_PASSWORD=root -d -p 3306:3306 mysql
```

Wait a few seconds for it to initialize.

Step 2: Enter MySQL CLI

```
docker exec -it mysql-test mysql -uroot -proot
```

Step 3: Run SQL Commands (directly in terminal)

```
CREATE DATABASE testdb;  
USE testdb;
```

```
CREATE TABLE users (  
  id VARCHAR(10) PRIMARY KEY,  
  name VARCHAR(50),  
  age INT  
);
```

Now Insert The values :

```
INSERT INTO users VALUES ('u001', 'Alice', 30);  
INSERT INTO users VALUES ('u002', 'Bob', 25);
```

Now Display the table contents:

```
SELECT * FROM users;
```

Output:

```
+-----+-----+-----+
| id    | name  | age  |
+-----+-----+-----+
| u001  | Alice | 30   |
| u002  | Bob   | 25   |
+-----+-----+-----+
```

```
[node1] (local) root@192.168.0.18 ~
$ docker run --name mysql-test -e MYSQL_ROOT_PASSWORD=root -d -p 3306:3306 mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
cea172a6e83b: Pull complete
fa811e9a869e: Pull complete
47a2982daa21: Pull complete
634d7076afe3: Pull complete
aa8a3958f09f: Pull complete
84e4e5ea3754: Pull complete
2275c0ff11a0: Pull complete
2792ea2d4e0e: Pull complete
f488b2cd8494: Pull complete
9451290759df: Pull complete
Digest: sha256:7839322bd6c3174a699586c3ea36314c59b61b4ce01b4146951818b94aef5fd7
Status: Downloaded newer image for mysql:latest
1b51eed0a878bf0907b1641e59fee6b43c019bf2d77afa76d0a033a65de23651
```

```
[node1] (local) root@192.168.0.18 ~
$ docker exec -it mysql-test mysql -uroot -proot
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 9.3.0 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql> CREATE DATABASE testdb;
Query OK, 1 row affected (0.004 sec)

mysql> USE testdb;
Database changed
mysql>
mysql> CREATE TABLE users (
    ->   id VARCHAR(10) PRIMARY KEY,
    ->   name VARCHAR(50),
    ->   age INT
    -> );
Query OK, 0 rows affected (0.008 sec)

mysql>
mysql> INSERT INTO users VALUES ('u001', 'Alice', 30);
Query OK, 1 row affected (0.012 sec)

mysql> INSERT INTO users VALUES ('u002', 'Bob', 25);
Query OK, 1 row affected (0.001 sec)
```

```
mysql>
mysql> SELECT * FROM users;
+-----+-----+-----+
| id    | name  | age  |
+-----+-----+-----+
| u001  | Alice | 30   |
| u002  | Bob   | 25   |
+-----+-----+-----+
2 rows in set (0.000 sec)

mysql> exit
Bye
[node1] (local) root@192.168.0.18 ~
$
```

11b. Programming example for PostgreSQL in Docker

Step 1: Start PostgreSQL

```
docker run --name pg-test -e POSTGRES_PASSWORD=root -d -p 5432:5432
postgres
```

Wait for it to initialize.

Step 2: Enter PostgreSQL CLI

```
docker exec -it pg-test psql -U postgres
```

Step 3: Run SQL Commands

```
CREATE TABLE users (  
    id TEXT PRIMARY KEY,  
    name TEXT,  
    age INT  
);
```

Now Insert the values:

```
INSERT INTO users VALUES ('P01', 'Ganesh', 30);  
INSERT INTO users VALUES ('P02', 'Suresh', 40);
```

Now, Display the table contents:

```
SELECT * FROM users;
```

Output:

id	name	age
P01	Ganesh	30
P02	Suresh	40

```
[node1] (local) root@192.168.0.18 ~
$ docker run --name pg-test -e POSTGRES_PASSWORD=root -d -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
8a628cdd7ccc: Pull complete
e4847368ad17: Pull complete
97cdd47d9131: Pull complete
2817206b0512: Pull complete
3a6f8814136c: Pull complete
07db60713289: Pull complete
0c942aac37b1: Pull complete
8c63b71925de: Pull complete
97f28320a07a: Pull complete
2a08aad74366: Pull complete
6cea4d95608f: Pull complete
c1b7de8085d1: Pull complete
f15c43cffa70: Pull complete
6948dc7760c1: Pull complete
Digest: sha256:fe3f571d128e8efadcd8b2fde0e2b73ebab6dbec33f6bfe69d98c682c7d8f7bd
Status: Downloaded newer image for postgres:latest
55df1cc9e8449c114f74b6e287861d0aaeb64b7d50adc4fe69f8e949191a149a
[node1] (local) root@192.168.0.18 ~
```

```
[node1] (local) root@192.168.0.18 ~
$ docker exec -it pg-test psql -U postgres
psql (17.4 (Debian 17.4-1.pgdg120+2))
Type "help" for help.
```

```
postgres=# DROP TABLE IF EXISTS users;
DROP TABLE
postgres=# exit
[node1] (local) root@192.168.0.18 ~
$
```

11c. MySQL in Docker : Complex Expt with multiple tables and 'select from where' queries

Step 1: Start MySQL Container

In your PWD terminal:

```
docker run --name mysql1 -e MYSQL_ROOT_PASSWORD=root -p 3306:3306
-d mysql:8
```

✓ This runs a MySQL 8 server named `mysql1`.

✓ `root` is the password.

Step 2: Connect to the Running MySQL

```
docker exec -it mysql1 mysql -uroot -proot
```

Now you'll see a prompt like:

```
shell
```

```
mysql>
```

Step 3: Create a New Database

Inside MySQL CLI type the following commands:

```
CREATE DATABASE shop;
```

```
USE shop;
```

Now Database 'shop' is created and selected.

Step 4: Create Two Tables

```
CREATE TABLE customers (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    city VARCHAR(100)
```

```
);
```

```
CREATE TABLE orders (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT,  
    product VARCHAR(100),  
    amount INT,  
    FOREIGN KEY (customer_id) REFERENCES customers(id)  
);
```

Note:1. Here `customers` and `orders` tables are created with **foreign key** relationships.

2. `AUTO_INCREMENT` automatically generates a **unique value** for a column when a new row is inserted — typically used for **primary key columns**, like `id`.

Step 5: Insert Some Sample Data

```
INSERT INTO customers (name, city) VALUES  
(  
    'Alice', 'New York',  
    'Bob', 'Chicago',  
    'Charlie', 'Los Angeles');
```



```
INSERT INTO orders (customer_id, product, amount) VALUES
(1, 'Laptop', 1200),
(1, 'Mouse', 25),
(2, 'Phone', 800),
(3, 'Tablet', 300),
(2, 'Keyboard', 45);
```

Step 6: Run Complex SELECT Queries

A. Find All Orders for "Alice"

```
SELECT o.id, o.product, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE c.name = 'Alice';
```

B. Find Orders With Amount Greater Than 500

```
SELECT c.name, o.product, o.amount
FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE o.amount > 500;
```

C. Find Customers Who Didn't Order Anything

```
SELECT name
FROM customers
WHERE id NOT IN (SELECT DISTINCT customer_id FROM orders);
```

D. Show Total Spend Per Customer

```
SELECT c.name, SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.id = o.customer_id
GROUP BY c.name
ORDER BY total_spent DESC;
```

Quick View of the Tables

If you ever want to **see table structure**:

```
DESCRIBE customers;
DESCRIBE orders;
```

And to **view all data**:

```
SELECT * FROM customers;
SELECT * FROM orders;
```

Deletion of Table entry:

```
DELETE FROM customers WHERE city = 'Chicago';
```

Dropping a table:

```
DROP TABLE orders;
```